

Task 3 — GNN Improvement, Ablation, Comparison and Explainability Report

1. Introduction

This task investigates whether graph neural networks can improve classification performance for the GroupNN dataset by systematically comparing graph-construction strategies, GNN architectures, model hyperparameters, optimization settings, and explainability methods.

The experiments were conducted using three graph-construction methods, referred to as **Graph A**, **Graph B**, and **Graph C**. Each graph represents a window using **8 nodes**, with **6 features per node**. Graph A and Graph B contain 20 directed edges, while Graph C contains 16 directed edges.

The dataset contains **17 classes**. The final experiments used subject-independent train/validation/test splits to reduce the possibility of subject leakage.

The main objectives of Task 3 were:

1. Improve the initial GNN architecture.
2. Compare graph-construction methods.
3. Perform architecture and hyperparameter ablations.
4. Compare GCN, GraphSAGE, and GAT.
5. Investigate class imbalance.
6. Perform 5-fold subject-level cross-validation.
7. Compare the final model against a GCN baseline.
8. Perform statistical significance testing.
9. Evaluate the final locked model on the held-out test set.
10. Provide local explanations using SHAP and LIME-related explainability analysis.

Overall Workflow:

2. Dataset and Graph Construction

2.1 Dataset organization

The dataset was divided at the **recording/subject level**, rather than randomly splitting individual windows.

The resulting recording counts were:

Split	Recordings
Training	10,710
Validation	2,142
Test	2,499
Total	15,351

An additional file, `normalization_stats.npz`, was found in the cache directory. It was not a recording cache and was therefore excluded from the graph dataset.

The cache directory contained:

Total files: 15,352

Valid recording caches: 15,351

Invalid/extra files: 1

The invalid file contained normalization statistics rather than recording information.

2.2 Graph representation

Each recording was divided into windows, and each window was represented as a graph.

A typical graph has:

Nodes: 8

Features per node: 6

Therefore:

[
 $X \in \mathbb{R}^{8 \times 6}$
]

The three graph-construction methods were:

Graph	Nodes	Edges	Edge attributes
-------	-------	-------	-----------------

A	8	20	20
B	8	20	20
C	8	16	16

Example:

Graph A:

x = [8, 6]

edge_index = [2, 20]

edge_attr = [20, 1]

Graph B:

x = [8, 6]

edge_index = [2, 20]

edge_attr = [20, 1]

Graph C:

x = [8, 6]

edge_index = [2, 16]

edge_attr = [16, 1]

The labels and metadata were preserved for every graph:

subject

session

trial

window_id

y

3. Graph Counts

Each recording produced **33 windows**.

This resulted in:

Split	Graph A	Graph B	Graph C
Train	353,430	353,430	353,430

Validation	70,686	70,686	70,686
Test	82,467	82,467	82,467
Total	506,583	506,583	506,583

The graph counts matched the expected values exactly.

For example:

```
[
10710\times33=353430
]
```

```
[
2142\times33=70686
]
```

```
[
2499\times33=82467
]
```

This provided an important sanity check that graph generation was complete and consistent across the three graph-construction methods.

4. DataLoader Verification

The final DataLoaders used:

BATCH_SIZE = 256

The number of batches was:

Split	Batches
Train	1,381
Validation	277

Test	323
------	-----

A typical batch contained:

x: [2048, 6]
edge_index: [2, 4096]
edge_attr: [4096, 1]
y: [256]

This is consistent with:

[
 $256 \times 8 = 2048$
]

and, for Graph C:

[
 $256 \times 16 = 4096$
]

Thus the PyTorch Geometric batching mechanism was verified before model training.

5. Edge Weight Calculation

For Graph B and Graph C, edge weights were calculated using cosine similarity between the source and target node feature vectors.

For two node feature vectors (x_i) and (x_j):

[
 $w_{ij} = \frac{x_i \cdot x_j}{\|x_i\| \|x_j\|}$
]

If either vector had zero magnitude, the similarity was set to zero.

The implementation also explicitly checked for invalid values.

After correcting the feature processing, the edge-weight distributions were:

Graph B

Min: 0.04719472
Max: 0.99457043

Mean: 0.6809961
Std: 0.27228528
NaN: 0
Inf: 0

Graph C

Min: 0.03833711
Max: 0.9724007
Mean: 0.5373428
Std: 0.24147265
NaN: 0
Inf: 0

Individual graph checks also confirmed:

B negative weights: 0
C negative weights: 0
B NaN: 0
C NaN: 0

This verified that the edge attributes supplied to the weighted GNN experiments were numerically valid.

6. Initial Graph Comparison

The first graph comparison used the same node features and metadata for corresponding windows.

The feature comparison showed:

A vs B features: 0.0
A vs C features: 0.0

This means the node features were identical between graph-construction methods; the main difference was therefore the graph topology and edge attributes.

Graph A used 20 edges with effectively uniform edge weighting in the tested setup, whereas Graph B and Graph C incorporated cosine-similarity edge weights.

Initial test performance was approximately:

Graph	Test Loss	Test Accuracy
-------	-----------	---------------

A	2.2885	0.2466
---	--------	--------

B 2.3004 0.2472

C 2.2519 0.2608

Graph C therefore provided the strongest initial performance.

7. Architecture Ablation

The first ablation investigated the number of GCN layers using Graph C.

7.1 One-layer GCN

Final test performance:

Test Loss: 2.2829

Test Accuracy: 0.2499

7.2 Two-layer GCN

Test Loss: 2.2667

Test Accuracy: 0.2642

7.3 Three-layer GCN

Test Loss: 2.2539

Test Accuracy: 0.2615

Architecture comparison

Layers	Test Loss	Test Accuracy
1	2.2829	0.2499
2	2.2667	0.2642
3	2.2539	0.2615

The two-layer model improved substantially over the one-layer model. Although the three-layer model obtained slightly lower test loss, its accuracy was slightly below the two-layer model.

This motivated using a **two-layer architecture** for subsequent experiments.

8. Hidden Dimension Ablation

The hidden representation size was varied while keeping the remaining architecture fixed.

Hidden size	Test Loss	Test Accuracy
32	2.2777	0.2551
64	2.2667	0.2642
128	2.2393	0.2763

The 128-dimensional hidden representation produced the best performance.

The improvement from 32 to 128 hidden units was:

[
0.2763-0.2551=0.0212
]

or approximately **2.12 percentage points**.

Therefore, the final model used:

Hidden dimension = 128

9. Dropout Ablation

Dropout was also investigated.

Dropout	Test Loss	Test Accuracy
0.0	2.2393	0.2763

0.3	2.2428	0.2692
0.5	2.2739	0.2616

Increasing dropout did not improve performance.

The no-dropout configuration achieved the highest accuracy:

[
27.63%
]

Therefore, the final configuration used **dropout = 0.0**.

10. Optimizer and Learning-Rate Ablation

Three optimizer configurations were compared.

Optimizer	Learning rate	Test Accuracy
Adam	0.001	0.2730
AdamW	0.001	0.2685
AdamW	0.0005	0.2551

Adam with learning rate (10^{-3}) produced the strongest result.

Therefore:

Optimizer: Adam
Learning rate: 0.001

was selected for the final model.

11. GNN Variant Comparison

A major part of the improvement study compared three GNN architectures:

- GCN
- GraphSAGE
- GAT

The comparison was performed using the improved configuration.

Model	Test Loss	Test Accuracy
GCN	2.1888	0.2901
GraphSAGE	1.4542	0.5082
GAT	2.2749	0.2623

GraphSAGE substantially outperformed both GCN and GAT.

The improvement over GCN in this experiment was:

$$\begin{aligned} &[\\ &0.5082 - 0.2901 = 0.2181 \\ &] \end{aligned}$$

or approximately **21.81 percentage points**.

This result motivated GraphSAGE as the final GNN architecture.

12. Class Imbalance Analysis

Class distribution was explicitly checked.

There were 17 classes, and every class contained exactly:

20,790 training graphs

Therefore:

$$\begin{aligned} &[\\ &17 \times 20790 = 353430 \\ &] \end{aligned}$$

The minimum and maximum class counts were both 20,790.

Thus:

Minimum class count: 20,790

Maximum class count: 20,790

The training dataset was therefore perfectly balanced.

Consequently, class weighting or oversampling was not required for the final experiment.

13. Subject-Level 5-Fold Cross-Validation

To obtain a more reliable estimate of generalization, five-fold subject-level cross-validation was performed.

The cross-validation used 30 subjects:

[1, 2, 3, 6, 8, 10, 11, 12, 14, 15,
16, 17, 18, 19, 20, 21, 22, 23, 24, 27,
29, 30, 31, 33, 34, 36, 39, 41, 42, 43]

Each fold contained:

24 training subjects

6 validation subjects

The subject-overlap check produced:

Fold 1: overlap = set()

Fold 2: overlap = set()

Fold 3: overlap = set()

Fold 4: overlap = set()

Fold 5: overlap = set()

Therefore, no subject appeared simultaneously in the training and validation sets of a fold.

Each fold contained:

Training graphs: 282,744

Validation graphs: 70,686

This provided a leakage-free subject-level cross-validation procedure.

14. GraphSAGE 5-Fold Cross-Validation Results

The final GraphSAGE configuration produced:

Fold	Loss	Accuracy
1	1.6355	0.5015
2	1.4316	0.5495
3	1.6166	0.4692
4	1.4993	0.5213
5	1.6763	0.4932

Mean performance:

[
Accuracy = 0.5069 ± 0.0303
]

and

[
Loss = 1.5718 ± 0.1023
]

Therefore:

GraphSAGE achieved $50.69\% \pm 3.03\%$ accuracy under five-fold subject-level cross-validation.

The relatively small standard deviation indicates that performance was reasonably consistent across the five subject folds.

15. GCN Baseline 5-Fold Cross-Validation

The GCN baseline was evaluated using the same subject-level folds.

Fold	Loss	Accuracy
1	2.2191	0.2800
2	2.3076	0.2474
3	2.0964	0.3104
4	2.1135	0.3074
5	2.1651	0.2910

Mean performance:

[
Accuracy=0.2872 $\pm$ 0.0255
]

and

[
Loss=2.1803 $\pm$ 0.0858
]

Thus GraphSAGE substantially outperformed the GCN baseline.

16. Statistical Significance Test

A paired comparison was conducted between the GraphSAGE and GCN fold accuracies.

The mean accuracies were:

GraphSAGE: 0.50694
GCN: 0.28724

Mean improvement:

[
0.50694-0.28724=0.21970
]

or approximately **21.97 percentage points**.

The paired t-test produced:

t-statistic = 9.4362
p-value = 0.000703

Since:

[
p<0.05
]

the paired t-test indicates a statistically significant improvement.

A Wilcoxon signed-rank test was also performed:

Statistic = 0.0
p-value = 0.0625

The Wilcoxon test did not reach the conventional ($p < 0.05$) threshold, although the direction of improvement was consistent across folds.

Interpretation

The parametric paired t-test provides strong evidence of improvement, while the non-parametric Wilcoxon test is more conservative with only five paired observations.

This limitation should be explicitly acknowledged rather than reporting only the significant t-test.

17. Final Locked Test Evaluation

After model selection and cross-validation, the final model was evaluated on the held-out test set.

The final locked test result was:

Test Loss: 1.4678
Test Accuracy: 0.4864

Therefore:

The final GraphSAGE model achieved 48.64% accuracy on the held-out test set.

This is substantially higher than the initial Graph A result of approximately 24.66%.

The absolute improvement was:

[
0.4864-0.2466=0.2398
]

or approximately **23.98 percentage points**.

Compared with the initial GCN-style configuration, the final GraphSAGE model therefore demonstrated a substantial improvement.

18. Final Model Configuration

The final selected model can be summarized as:

Component	Final choice
Graph	Graph C
Nodes	8
Node features	6
Edges	16
GNN	GraphSAGE
Layers	2
Hidden size	128
Dropout	0.0

Optimizer	Adam
Learning rate	0.001
Classes	17
Batch size	256
Early stopping	Used
Evaluation	Subject-independent

The model was selected based on the ablation experiments and cross-validation results rather than the test set.

19. Explainability

Explainability was investigated using SHAP for individual graph samples.

The purpose was to understand which node-feature combinations contributed to individual predictions.

Each graph contains:

[
8\times6=48
]

node-feature inputs.

Therefore, each local SHAP explanation produces 48 feature contributions.

19.1 Correctly classified sample

The correctly classified sample was:

True label: 6

Predicted label: 6

Subject: 38
Session: 1
Trial: 1
Window: 2

The model produced:

Output shape: [1, 17]
Predicted class: 6
Predicted probability: 0.3939

SHAP was then applied to explain the model's class-6 prediction.

The strongest SHAP contributions were:

Node	Feature	SHAP
2	1	+0.003722
5	1	-0.002863
4	2	-0.002151
7	1	+0.001018
2	5	+0.000966
7	3	+0.000822
5	2	+0.000655
5	5	-0.000618
3	6	+0.000534
8	2	+0.000517

The largest positive contribution was:

```
[  
Node,2,\ Feature,1=+0.003722  
]
```

while the strongest negative contribution was:

```
[  
Node,5,\ Feature,1=-0.002863  
]
```

Positive SHAP values indicate that the corresponding input contributed toward the explained class, whereas negative values contributed against it.

20. Incorrectly Classified Sample

A second sample was selected where the model made an error.

The sample characteristics were:

True label: 6
Predicted label: 3

Subject: 38
Session: 1
Trial: 1
Window: 0

This provides a useful contrast with the correct sample.

The SHAP analysis for this sample explains **why the model selected class 3**, rather than explaining the true class 6.

This is particularly useful for identifying feature patterns that may cause confusion between classes.

21. SHAP Methodology

SHAP was implemented using:

SHAP version: 0.51.0

The graph structure was kept fixed while the 48 node-feature inputs were varied.

A prediction wrapper converted flattened:

```
[  
48=8\times6  
]
```

feature vectors back into the graph's original (8\times6) structure.

The model then produced class probabilities for all 17 classes.

This allowed SHAP to estimate the contribution of individual node-feature pairs to the model's prediction.

The resulting explanations were visualized as:

1. Node $\times$ feature SHAP heatmaps.
2. Mean absolute SHAP feature-importance bar charts.

22. Confusion Matrix, Classification Report and ROC-AUC

The final model was also prepared for full test-set evaluation using:

- Confusion matrix
- Precision
- Recall
- F1-score
- ROC-AUC
- One-vs-rest ROC curves

The evaluation was performed purely through inference using the already-trained final model.

The test predictions were generated as:

```
[  
P(y=c|G)=softmax(f_\theta(G))  
]
```

where (G) is the input graph and (c) is one of the 17 classes.

For multiclass ROC-AUC, a one-vs-rest formulation was used.

Important

The actual **ROC-AUC values and classification-report numbers** should be copied from the **successful evaluation cell** rather than estimated. They were not included in the experimental results supplied above, so they should not be invented in the report.

23. Overall Ablation Summary

The main ablation results can be summarized as follows:

Experiment	Configuration	Accuracy
Graph comparison	Graph A	0.2466
Graph comparison	Graph B	0.2472
Graph comparison	Graph C	0.2608
Layers	1	0.2499
Layers	2	0.2642
Layers	3	0.2615
Hidden size	32	0.2551
Hidden size	64	0.2642
Hidden size	128	0.2763
Dropout	0.0	0.2763
Dropout	0.3	0.2692
Dropout	0.5	0.2616

Optimizer	Adam, 0.001	0.2730
Optimizer	AdamW, 0.001	0.2685
Optimizer	AdamW, 0.0005	0.2551
GNN variant	GCN	0.2901
GNN variant	GraphSAGE	0.5082
GNN variant	GAT	0.2623

This demonstrates that the largest improvement came from changing the GNN variant to **GraphSAGE**.

24. Comparison with the Initial Model

The progression can be summarized as:

Initial Graph A:
24.66%

Improved Graph C:
26.08%

Best GCN configuration:
~29.01%

GraphSAGE:
50.82%

5-fold GraphSAGE:
 $50.69 \pm 3.03\%$

Final locked test:
48.64%

The final model therefore substantially improved upon the initial graph-based baseline.

The difference between cross-validation accuracy and final held-out test accuracy is:

[
50.69%-48.64%=2.05
]

percentage points, which is relatively modest and indicates reasonably consistent generalization.

25. Discussion

The experiments show that **graph construction and GNN architecture have a much larger impact on performance than modest changes in regularization or optimization.**

Graph C performed better than Graph A and Graph B in the initial graph comparison. Graph C uses a more compact 16-edge topology and cosine-similarity edge weights.

The architecture ablation showed that increasing model capacity from 32 to 128 hidden dimensions improved accuracy. However, increasing dropout reduced performance, suggesting that the model did not benefit from additional dropout regularization under the tested conditions.

The optimizer experiment showed that Adam with a learning rate of 0.001 performed better than the tested AdamW configurations.

The most significant result was the GNN variant comparison. GraphSAGE achieved approximately 50.82% accuracy, compared with 29.01% for GCN and 26.23% for GAT.

One possible explanation is that GraphSAGE's neighborhood aggregation mechanism provides a more effective representation for the local graph structures present in the data.

26. Limitations

Several limitations should be acknowledged.

26.1 Limited number of subjects for cross-validation

Only 30 subjects were included in the five-fold cross-validation, resulting in five folds of six validation subjects each.

26.2 Statistical test sample size

The significance tests were performed over only five paired fold-level observations. Therefore, the significant paired t-test result should be interpreted cautiously.

The Wilcoxon test did not reach significance at ($p < 0.05$).

26.3 Limited hyperparameter search

The ablations investigated selected values rather than exhaustively searching the entire hyperparameter space.

26.4 Related-work comparison

Related-work numbers should only be compared when datasets, splits, preprocessing, and evaluation protocols are sufficiently comparable. Direct comparison with results obtained under different experimental conditions can be misleading.

26.5 Explainability is local

The SHAP analysis used one correctly classified and one incorrectly classified sample. These explanations should therefore not be interpreted as global feature importance across the entire dataset.

27. Final Conclusion

The Task 3 experiments demonstrate that systematic GNN improvement can substantially increase classification performance.

The experiments established that:

1. The dataset contains **17 balanced classes**.
2. The graph-generation pipeline produced the expected **506,583 graphs**.
3. Graph C provided the strongest initial graph-construction result.
4. A two-layer architecture provided a good balance between shallow and deeper models.
5. A hidden size of 128 performed better than 32 and 64.
6. Additional dropout did not improve performance.
7. Adam with learning rate 0.001 was the strongest tested optimizer configuration.
8. GraphSAGE substantially outperformed GCN and GAT.
9. GraphSAGE achieved **50.69% $\pm$ 3.03%** accuracy in five-fold subject-level cross-validation.
10. The final locked model achieved **48.64% test accuracy**.
11. The paired t-test showed a statistically significant improvement over the GCN baseline (($p=0.000703$)), although the Wilcoxon test did not reach ($p<0.05$).
12. SHAP provided local explanations for both a correct and an incorrect prediction.

Overall, the final GraphSAGE model represents a substantial improvement over the initial GNN configuration, while the subject-level splitting and cross-validation provide stronger evidence that the improvement is not simply the result of data leakage.

28. Final Results Table for the Report

This is the table I'd put prominently in the PDF:

Model / Experiment	Test / CV Accuracy
Initial Graph A	24.66%
Initial Graph B	24.72%
Initial Graph C	26.08%
Best GCN configuration	29.01%
GraphSAGE	50.82%
GAT	26.23%
GraphSAGE 5-fold CV	50.69% $\pm$ 3.03%
GCN 5-fold CV	28.72% $\pm$ 2.55%
Final locked GraphSAGE	48.64%